

Komponenten

Tim Schmitz
05.07.2023

Inhalt

Definition	3
Hauptkomponente	3
Decorator	3
Template	4
Styles	4
Deklaration	5

Komponenten

Komponenten definieren Ansichten (Views) für die Anwendung. Komponenten definieren also, was der Benutzer von der Anwendung sieht. Technisch gesehen werden mit Komponenten neue HTML-Tags definiert.

Definition

Eine Komponente besteht aus drei Teilen:

1. Eine Klasse mit dem Component-Decorator. Dieser definiert die Klasse zu einer Komponente.
2. Ein Template, das im Grunde das HTML der Komponente enthält.
3. Die Styles der Komponente, die über CSS-Stylesheets erzeugt werden können.

Die Klasse und das Template sind erforderlich, die Styles dagegen sind optional. Es gibt zwei Möglichkeiten, das Template und die Styles zu definieren.

- Sie können direkt im Decorator als inline-Deklarationen angegeben werden
- Sie können auch in eigenen Dateien definiert werden und dann im Decorator referenziert werden

In dem letzteren Fall sollten die Komponenten dieselben Dateierweiterungen wie die Dateien mit den entsprechenden Klassen besitzen:

Komponente	Namen
Klasse	app.component.ts
Template	app.component.html
Style	app.component.css

Tabelle 1: Bezeichner für Komponenten

Hauptkomponente

Die Hauptkomponente wurde von der Angular-Entwicklungsumgebung erzeugt und besitzt folgendes Aussehen:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'The Contacts-Manager';
}
```

Diese Klasse definiert Eigenschaften und Methoden. Diese können im Template referenziert werden und zum Beispiel dem Benutzer den Wert einer Eigenschaft anzeigen. Standardmäßig wird in der Hauptkomponente nur die Eigenschaft "title" angezeigt.

Decorator

Der Component-Decorator wird mit dem Schlüsselwort @Component deklariert und besitzt die folgenden Eigenschaften:

selector

Diese Eigenschaft definiert den Tag-Namen (app-root), mit dem die Komponente im Template (app.component.html) referenziert wird.

templateUrl

Diese Eigenschaft definiert das Template der Komponente. Normalerweise steht hier ein relativer Pfad zu einer HTML-Datei. Wird hingegen nur die Eigenschaft "template" angegeben, so folgt ein vollqualifizierter HTML-Pfad. Diese Eigenschaft ist die einzige, die nicht optional ist.

styleUrls

Diese Eigenschaft definiert den Style der Komponente. Hier steht ein Array von Strings, jeder String ist ein relativer Pfad zu einer CSS-Datei.

providers

Dieser Array registriert Providers für die Komponente und alle ihre Kinder.

viewEncapsulation

Diese Eigenschaft kapselt die Auswirkung von CSS-Styles auf bestimmte Bereiche.

Template

Das Template der Komponente ist eine HTML-Datei. Sie definiert, was der Nutzer zur Laufzeit sieht und wie der Nutzer mit dieser interagiert. Das Template schreibt man mit HTML und weiteren Syntaxelementen, die in Angular zur Verfügung stehen.

Zum Beispiel gibt es im normalen HTML keine Möglichkeit aus einem Template eine Eigenschaft aus TypeScript zu referenzieren. In Angular geschieht dies über das "Data Binding" (Datenbindung). Damit werden Eigenschaften und Methoden im Template nutzbar.

Ein Ausschnitt des Templates lautet zum Beispiel:

```
<div class="card highlight-card card-small">
  <span>{{ title }} app is running now!</span>
  ...
</div>
```

Die Datenbindung lautet damit {{title}}. Damit sieht der Benutzer zur Laufzeit den Wert der Eigenschaft title aus der Komponente. Diese Eigenschaft und deren Wert sind in der Klasse der Komponente definiert.

```
export class AppComponent {
  title = 'The Contacts-Manager';
}
```

Falls sich der Wert von title zur Laufzeit ändert, dann wird auch die Ansicht entsprechend angepasst. Abgesehen davon besteht der Rest des Templates aus regulärem HTML. Diese Art der Datenbindung heißt Interpolation, die sich am besten für Strings und Zahlen als Werte eignet.

Der Prozess, der auf die Datenbindung reagiert, heißt in Angular Change Detection. Er überprüft, ob sich Daten geändert haben. Der Prozess selbst wird vom Webserver durchgeführt, sobald ein Ereignis eintritt.

Styles

Jede Komponente kann ihre eigenen Styles besitzen. Diese stehen in einer (leer generierten) Datei `app.component.css`. Die globalen Styles stehen in der Hauptdatei `styles.css`. Es macht Sinn die Styles zu lokalisieren, um die globale Datei zu entlasten und die Auswirkungen zu limitieren. Referenziert werden die Styles in dem Stylesheet-Pfad des Decorators unter der Eigenschaft `styleUrls`.

Es gibt folgende Vorteile, Styles für eine Komponente zu definieren:

- Es gibt weniger globale Styles und somit auch eine kleinere globale Datei `styles.css`
- Komponenten-lokale Styles haben keine Auswirkungen auf andere Komponenten

Diese Einschränkungen können durch ViewEncapsulation geändert werden.

- None: Die Styles werden global angewendet, man kann aber externe Bibliotheken wie Bootstrap einbinden
- Emulated: Das ist die Standardeinstellung, in der die Styles nur lokal angewendet werden, globale Styles allerdings auch
- ShadowDom: Die Styles wirken sich nur auf die definierende Komponente aus

Deklaration

Jede Komponente muss deklariert werden, d.h. die Klasse der Komponente muss im `declarations`-Array des Angular-Modul stehen. Ansonsten würde es zur Laufzeit zu einer Exception kommen. Das Code-Listing des Modules sieht wie folgt aus:

```
import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  ...
})
export class AppModule { }
```

Damit gehört die Komponente zum Module. Die Komponente darf nur in genau einem Angular-Modul deklariert werden.